

יחידה 5: רשתות קונבולוציה (CNN)

מוטיבציה לשימוש ברשתות קונבולוציה

ביחידות הלימוד הקודמות למדנו כיצד לאמן רשת נוירונים לסיווג תמונות פריטי הלבוש באוסף הנתונים Fashion-MNIST, אך לא השקענו מחשבה רבה בבחירת ארכיטקטורת הרשתות שאימנו. דבר זה לא היה בעיה שכן שכבות הרשת תמיד היו בעלות קישוריות מלאה (FC – Fully Connected), כלומר כל נוירון בהן היה מחובר לכל נוירון בשכבה העוקבת. שכבות אלו בעלות כוח הבעה רב, ובחירה מתאימה של משקלי הרשת אפשרה להן ללמוד כללים המפרידים בין מחלקות פריטי הלבוש. עם זאת, לכוח הבעה פוטנציאלי זה יש מחיר: השימוש בשכבות אלו יקר מבחינה חישובית, וכן יש בהן מספר רב של פרמטרים כך שדרוש סט אימון גדול במיוחד כדי לאמן. כל עוד הבעיה שאנו עוסקים בה פשוטה דיה, כגון סיווג תמונות שחור-לבן בגודל 28×28 פיקסלים ל-10 מחלקות, מחיר זה לא בא לידי ביטוי, אך מהר מאוד, עם העלייה במורכבות הבעיה, המחיר נעשה בלתי ריאלי. חשבו לדוגמה על רשת אשר הקלט שלה הוא תמונות צבעוניות בגודל 1 מגה־פיקסל: ממד הקלט הוא 3 מיליון (מיליון פיקסלים לכל אחד משלושת ערוצי הצבע), ולכן **לכל נוירון** בשכבה השנייה יהיו 3 מיליון משקלים. אפילו אם השכבה השנייה תהיה קטנה והרשת לא עמוקה במיוחד, העלות החישובית של מעבר קדימה יחיד תהיה גבוהה, ועל אחת כמה וכמה העלות של תהליך האופטימיזציה.

כוח ההבעה הרב של שכבות FC מאפשר גמישות, אך לעתים שלא לצורך: חשבו למשל על כך שבכל הרשתות אשר אימנו עד כה, מיקום הפיקסלים בתמונה לא הובא בחשבון. ייתכן שעם תהליך האימון הרשת למדה שקיימת קורלציה בין ערכיהם של פיקסלים מסוימים, אך היא יכלה ללמוד זאת גם אילו היה הקלט "מעורבב" לפני הזנתו, וזאת מפני שבתחילת תהליך האימון שיטחנו את טנזור הקלט, וכל איבר של טנזור הקלט השפיע באותה המידה על החישוב העוקב.

לצורך דוגמה, נגריל תמורה אקראית על שורות התמונות,

```
row_mix = torch.randperm(28)
print(row_mix)
```

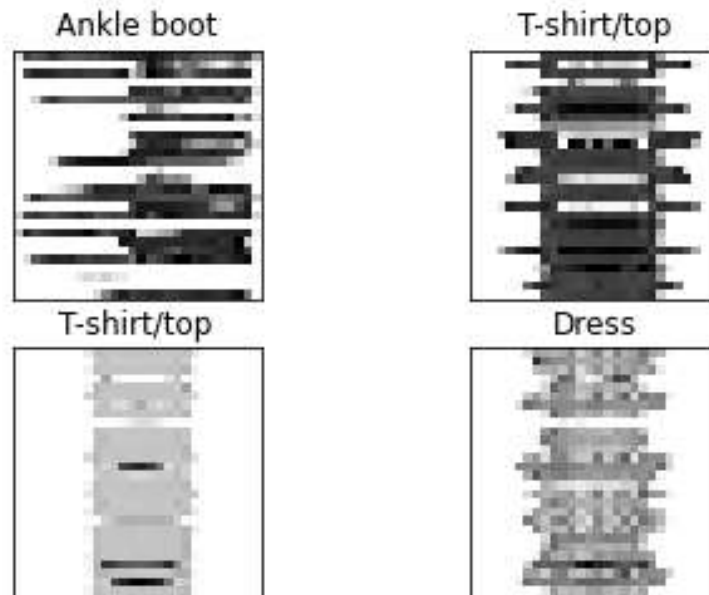
פלט:

```
tensor([21,  6, 18,  0, 11, 23,  4, 12, 27,  9,  7, 13, 24,
        3,  5, 16, 22,  8, 20, 26, 19, 14, 10, 17,  1, 25,  2, 15])
```

ונפעיל תמורה זו על השורות של כל התמונות בסט האימון כדלקמן.

```
imgs = imgs[:, :, row_mix, :]
```

זכרו שהמשתנה `imgs`, הנטען דרך ה-`dataloader`, מכיל `batch` של תמונות ועל כן הוא טנזור 4-ממדי: הממד הראשון הוא ממד ה-`batch`, השלישי והרביעי הם ממדי השורה והעמודה של התמונה (ועל כן גודל כל אחד מהם הוא 28). הממד השני הוא ממד הערוץ (`channel`), אך התמונות באוסף נתונים זה הן בשחור-לבן ולכן בעלות ערוץ יחיד בלבד. תמונות צבעוניות לרוב יכילו שלושה ערוצים עבור עוצמת האדום, הירוק והכחול בפיקסל הנתון. נפעיל את התמורה על 4 פריטי לבוש מקריים ונאייר זאת להלן.



כאשר פריטי הלבוש המקוריים הם:



מהאיורים העליונים כלל לא ברור לעין אנושית באילו פריטי לבוש מדובר, אולם רשתות הנוירונים שעסקנו בהן עד כה יפגינו ביצועים זהים עבור כל אחת מצורות הקלט, כל עוד נפעיל את אותה הטרנספורמציה על כל התמונות באוסף נתוני האימון והבדיקה.

ביחידה הנוכחית נוותר על עוצמה זו, אך בתמורה נקבל שכבות יעילות הרבה יותר, המותאמות ספציפית למשימות העוסקות בעיבוד תמונה וזיהוי פרטים. יעילות זו תאפשר לנו לאמן רשתות עמוקות יותר, אשר לבסוף, תמורת אותו מחיר חישובי, יבצעו עבודה טובה הרבה יותר מרשתות בעלות קישוריות מלאה. הרשתות החדשות יורכבו משכבות קונבולוציה (convolution), השואבות השראה ממערכת הראייה האנושית ומשיטות קלאסיות של עיבוד תמונה דיגיטלי. שכבות קונבולוציה מסכמות את השכבה הקודמת באמצעות חילוץ מאפיינים פשוטים (כגון קיום פינה במיקום מסוים בתמונה). באמצעות שרשרת היררכי של השכבות זו לזו, מחולצים לעומק הרשת מאפיינים מורכבים יותר של התמונה המקורית (כגון קיומם של שרוול חולצה או שרוכי נעליים). לבסוף, על בסיס המאפיינים שהרשת למדה לחלץ מהתמונה, יתבצע הסיווג למחלקות השונות.

בהמשך יחידה זו תלמדו את הפרטים המדויקים של פעולת רשת כזו, את הרכיבים השונים שהיא מורכבת מהם, וכמובן את מימושם בקוד ושילובם באלגוריתם למידה מבוסס גרדיאנט.

שאלות לתרגול

1. א. אמנו רשת עמוקה בעלת קישוריות מלאה לחיזוי המחלקות של אוסף הנתונים Fashion-MNIST לאחר שהגרלתם תמורה אקראית **לכל הפיקסלים** בתמונה והפעלתם אותה על כל תמונה לפני הזנתה.
 - ב. בדקו את ביצועי הרשת על סט הבדיקה לאחר שעל כל תמונה בו הופעלה אותה תמורה.
 - ג. בדקו את ביצועי הרשת על סט הבדיקה כאשר לא הופעלה על התמונות בו כל תמורה.
2. אמנו שוב את הרשת כפי שעשיתם בשאלה 1, אך כעת הגרילו מחדש תמורה אקראית עבור כל minibatch. הסבירו את התוצאה שהתקבלה.

פעולת הקונבולוציה עם גרעין

הפעולה הבסיסית המשמשת לבניית שכבות קונבולוציה אינה המצאה חדשה של מומחי הלמידה העמוקה, אלא פונקציה ידועה בתחום עיבוד התמונה – מדובר בקונבולוציה של תמונת הקלט עם גרעין (kernel). בתחום עיבוד התמונה פעולה זו נקראת לעתים הפעלת פילטר, ולה מגוון שימושים שונים, כגון טשטוש תמונה, ניקוי רעש או זיהוי קצוות של אובייקטים.

גרעין הקונבולוציה הוא מטריצה קטנה, אשר ערכיה נקבעים בהתאם לאפקט הרצוי על תמונת הפלט. למשל השימוש במטריצה

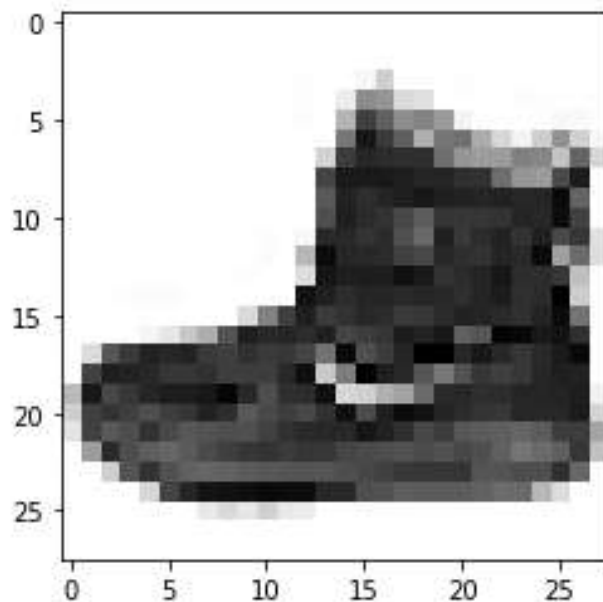
$$K = \begin{pmatrix} \frac{1}{16} & \frac{1}{8} & \frac{1}{16} \\ \frac{1}{8} & \frac{1}{4} & \frac{1}{8} \\ \frac{1}{16} & \frac{1}{8} & \frac{1}{16} \end{pmatrix}$$

יוביל לטשטוש התמונה, כפי שנראה מייד. להשלמת הדוגמה נתבונן באחת התמונות מהאוסף Fashion-MNIST – תמונה בגודל 28×28 פיקסלים אשר כל אחד מהם מיוצג באמצעות מספר טבעי בטווח 0 (לבן) עד 255 (שחור), כמפורט בקטע הקוד שלהלן.

```
img, _ = next(iter(train_dataloader))
img = torch.squeeze(img)
print(img.dtype, img.size(), sep='\n')
print(img[9:15, 9:15])
plt.imshow(img, cmap='Greys');
```

פלט:

```
torch.uint8
torch.Size([28, 28])
tensor([[ 0,  0,  0,  0, 183, 225],
        [ 0,  0,  0,  0, 193, 228],
        [ 1,  3,  0, 12, 219, 220],
        [ 0,  6,  0, 99, 244, 222],
        [ 4,  0,  0, 55, 236, 228],
        [ 0,  0,  0, 237, 226, 217]], dtype=torch.uint8)
```



נגדיר בזיכרון את גרעין הקונבולוציה הנ"ל, ואחר כך נחשב את הקונבולוציה של תמונת המגף עם גרעין זה.

```
kern = torch.tensor([[1/16, 1/8, 1/16],
                      [1/8, 1/4, 1/8],
                      [1/16, 1/8, 1/16]])
```

כעת נסרוק את המטריצה של תמונת הקלט משמאל לימין ומלמעלה למטה, ועבור כל תת-מטריצה מסדר 3×3 נחשב פיקסל יחיד של תמונת הפלט באמצעות:

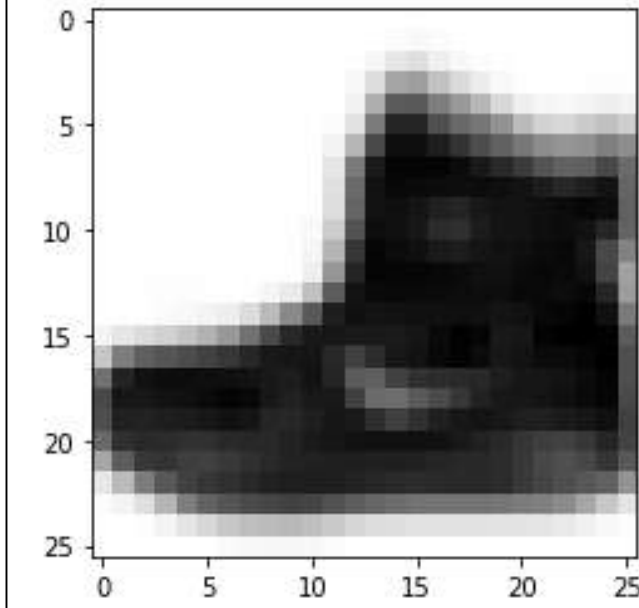
1. כפל חלק התמונה הרלוונטי בגרעין, איבר-איבר.
2. סכימת התוצאה.

```

output = torch.empty(26, 26)
for i in range(26):
    for j in range(26):
        sub_img = img[i:i+3, j:j+3]
        output[i, j] = (sub_img * kern).sum()
plt.imshow(output, cmap='Greys');

```

פלט:



שימו לב שבקוד הנ"ל המשתנים i, j מסמנים את הפינה השמאלית העליונה של תת-המטריצה sub_img . לפיכך, ערכם המרבי הוא 25, בעוד אינדקס השורה או העמודה המרבי של תמונת הקלט הוא 27. אין זו טעות, שכן כדי ליצור את תת-המטריצה sub_img יש צורך בנוכחותם של עוד שני איברים מימין ומתחת ל- i, j במטריצת הקלט. ניתן לראות אפוא שפלט פעולת הקונבולוציה הוא תמונה קטנה יותר, בגודל 26×26 . לעתים הקטנת הממד תהיה שימושית עבורנו, ולעתים נרצה לשמור על גודל התמונה המקורי. בהמשך היחידה נדון בשיטות לשליטה בממד הפלט.

אין זה הכרחי שגרעין הפעולה יהיה ריבועי, ראו למשל דוגמה נוספת שבה נבצע קונבולוציה של התמונה עם הגרעין $K = \begin{pmatrix} -1 & 1 \end{pmatrix}$:

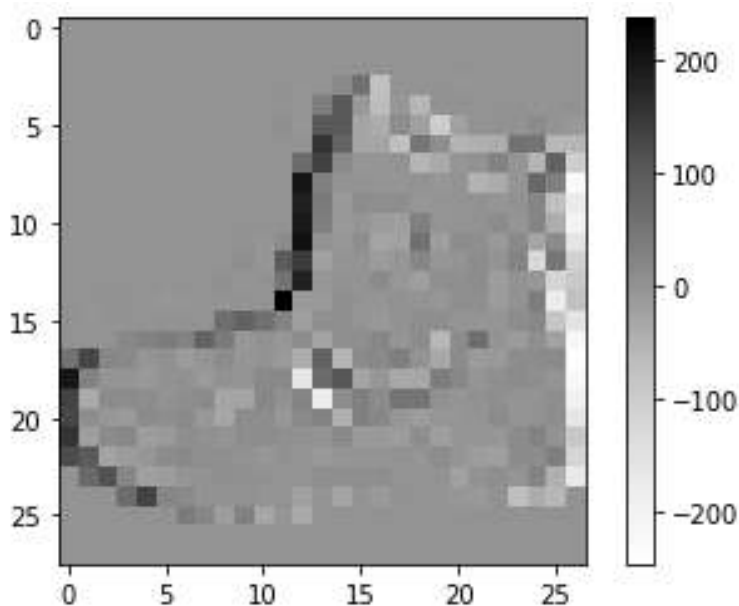
```

kern = torch.tensor([-1,1])
output = torch.empty(28,27)
for i in range(28):
    for j in range(27):
        sub_img = img[i,j:j+2]
        output[i,j] = (sub_img*kern).sum()
print(output.size())
plt.imshow(output, cmap='Greys');
plt.colorbar();

```

פלט:

```
torch.Size([28, 27])
```



לעיל ניכר כי רוב מטריצת הפלט מלאה בערכים סביב האפס (אפור), אך יש ערכים גבוהים במקומות שבתמונה המקורית היה מעבר מלבן לשחור (בתנועה משמאל לימין) וערכים נמוכים במעבר הפוך. לאור זאת ניתן להגיד שהגרעין הנ"ל הוא למעשה מזהה קצוות אופקי. ראו גם כי פלט פעולת הקונבולוציה הפעם הוא מממד 28×27 , שכן לא בוצע כל חישוב עבור העמודה האחרונה בתמונה המקורית: מימין לעמודה זו אין פיקסלים שבעזרתם ניתן להגדיר את תת-המטריצה `sub_img`.

מדוגמאות אלו נקיש על הכלל. בהינתן מטריצת קלט מסדר $H \times W$ (המייצגת תמונה בעלת ערוץ צבע יחיד: שחור-לבן),

$$X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,W} \\ \vdots & \ddots & \vdots \\ x_{H,1} & \cdots & x_{H,W} \end{pmatrix}$$

ומטריצה קטנה מסדר $p \times q$, אשר תשמש כגרעין,

$$K = \begin{pmatrix} k_{1,1} & \cdots & k_{1,q} \\ \vdots & \ddots & \vdots \\ k_{p,1} & \cdots & k_{p,q} \end{pmatrix}$$

תוצאת פעולת הקונבולוציה של X עם K היא מטריצה Y מסדר $(H - p + 1) \times (W - q + 1)$ בעלת הערכים האלה:

$$y_{r,s} = \sum_{i=1}^p \sum_{j=1}^q x_{r+i-1,s+j-1} k_{i,j}$$

שימו לב שלפי נוסחה זו, כדי לחשב את $y_{r,s}$, יש:

1. לחתוך מ- X תת-מטריצה מסדר $p \times q$ אשר הפינה השמאלית העליונה שלה היא $x_{r,s}$,
2. לכפול תת-מטריצה זו איבר-איבר במטריצת הגרעין,
3. לסכום את התוצאה.

מהגדרת הפעולה נבין גם את ממד הפלט: ניתן לחשב את $y_{r,s}$ כל עוד מתחת ל- $x_{r,s}$ יש עוד $p - 1$ שורות ומימין לו $q - 1$ עמודות.

לסיום פרק זה נציין שבעוד בתחומי עיבוד התמונה והלמידה העמוקה הפעולה המתוארת לעיל קרויה קונבולוציה, שמה המתמטי הוא קורלציה צולבת (cross-correlation), והפעולה המתמטית הקרויה קונבולוציה שונה במעט. למרות זאת נמשיך, כמקובל בתחום, להשתמש בשם קונבולוציה עבור הפעולה הרלוונטית לשימושנו.

שאלות לתרגול

1. כתבו פונקציית פייתון המקבלת מטריצת קלט וגרעין כלשהו ומחשבת את הקונבולוציה שלהם. על הפונקציה לבדוק את תקינות הקלט, כלומר שבכלל ניתן לבצע את הקונבולוציה (יש לבדוק שגרעין הקונבולוציה אינו גדול מדי).
2. תכננו גרעין 3×3 המזהה קצוות אלכסוניים והפעילו אותו על התמונה של המגף הנ"ל.

מבנה שכבת קונבולוציה ברשתות נוירונים

את פעולת הקונבולוציה אשר הכרנו בפרק הקודם נרתום כעת עבור רשתות נוירונים בעלות קלט ויזואלי. לשם כך, הצעד הראשון שננקוט הוא הפיכת גרעין הקונבולוציה לפרמטרים נלמדים של רשת הנוירונים: בעוד למטרות עיבוד תמונה אנו מגדירים את גרעין הקונבולוציה מראש עבור שימוש ידוע, כאשר נשלב פעולה זו בלמידה עמוקה נניח לערכי מטריצת הגרעין להיקבע לפי אלגוריתם האופטימיזציה. משמעות הדבר היא שהרשת תלמד בעצמה גרעינים המחלצים מאפיינים רלוונטיים למטרותיה. אם כן, נתחיל כמנהגנו בהגדרה ראשונית של שכבת קונבולוציה, ונמשיך ונשפר אותה בהמשך היחידה.

נניח כי הקלט לשכבה הוא minibatch בעל N דגימות של תמונות בגודל $H \times W$, שלהן ערוץ צבע יחיד. לפיכך, גודל טנזור הקלט יהיה $N \times H \times W$, כאשר הקונבולוציה תופעל בנפרד על כל תמונה ב־minibatch.

```
class ConvLayer_1(nn.Module):
    def __init__(self, kernel_size):
        super().__init__()
        self.kern = nn.Parameter(torch.rand(kernel_size))
        self.p, self.q = kernel_size
    def forward(self, X):
        output = torch.empty(X.size(0),
                              X.size(1)-self.p+1,
                              X.size(2)-self.q+1)
        for i in range(output.size(1)):
            for j in range(output.size(2)):
                sub_img = X[:, i:(i+self.p), j:(j+self.q)]
                output[:, i, j] = (sub_img*self.kern).sum(dim=(1,2))
        return output
```

במימוש זה בקוד מופיעים כמה פרטים מעניינים:

- ראשית שימו לב שגרעין הקונבולוציה מאותחל באקראי ומוגדר כאובייקט `nn.Parameter` שכן הוא ישתתף בתהליך האימון, ובעת ביצוע צעד של אובייקט האופטימיזציה נרצה שגם הוא יעודכן.
- שנית, ראו שבעת חישוב האיבר ה־ i, j של הפלט, הגרעין משודר לאורך ממד ה־batch של תת־התמונה, ואחרי כן מתודת הסכום מבצעת הפחתה על ממד הגובה והרוחב של התמונה, כך שנשאר סקלר יחיד עבור כל דגימה ב־batch.
- לבסוף, שימו לב לגמישות המובנית בתוך השכבה: בניגוד לשכבות לינאריות, בנאי שכבה זו לא דורש את ממד הקלט והפלט מראש. בעת המעבר קדימה בשכבה ייקבע ממד הפלט לפי חוקיות הקונבולוציה. משמעות הדבר היא שניתן להשתמש בשכבה מסוג זה עבור קלט המשתנה בגודלו.

הפרט המעניין ביותר בשכבת הקונבולוציה הוא שבעת המעבר קדימה אנו מבצעים רק פעולות כפל וחיבור בין הגרעין לבין איברי מטריצת הקלט, ולכן בעת התפשטות הגרדיאנט לאחור לא תתעורר בעיה – שהרי אלו פעולות גזירות (הגם שמערכת ה־autograd תטפל בכך עבורנו).

נדגים זאת באמצעות אתחול שכבת קונבולוציה ואימונה באמצעות SGD לזהות קצוות אנכיים: נגדיר פילטר זיהוי קצוות בעל הגרעין

$$K = \begin{pmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{pmatrix}$$

בעזרת המחלקה שכתבנו זה עתה:

```
edge_detector = ConvLayer_1((3,3))
edge_detector.kern.requires_grad = False
edge_detector.kern[:] = torch.tensor([[-1., -1, -1],
                                       [0, 0, 0],
                                       [1, 1, 1]])
```

ראו כי כיבנו את מעקב מערכת ה־autograd על גרעין מזהה הקצוות, שכן הוא יהיה המטרה שאליה שכבת הקונבולוציה שנגדיר להלן תתאים את עצמה לאורך תהליך האופטימיזציה.

```
my_conv = ConvLayer_1((3,3));
optimizer = torch.optim.SGD(my_conv.parameters(), lr=0.1)
```

כל שנוטר לעשות הוא לכתוב את לולאת האימון, שבה נטען minibatch של תמונות מהאוסף Fashion-MNIST, ננרמל אותן למספרים ממשיים בטווח 0 עד 1 ונעביר אותן דרך פילטר זיהוי הקצוות ושכבת הקונבולוציה שלנו. המרחק בין שתי התוצאות ישמש כפונקציית ההפסד של האופטימיזציה, וכך גרעין השכבה יעודכן לאורך התהליך לייצר פלט דומה לזה של edge_detector. לאחר טעינת התמונות לתוך המשתנה imgs והמרתן לטנזורים מנורמלים, ליבת לולאת האימון תיראה כך:

```
optimizer.zero_grad()
my_transformed_imgs = my_conv(imgs)
target_imgs = edge_detector(imgs)
loss = ((my_transformed_imgs-target_imgs)**2).mean()
loss.backward()
optimizer.step()
```

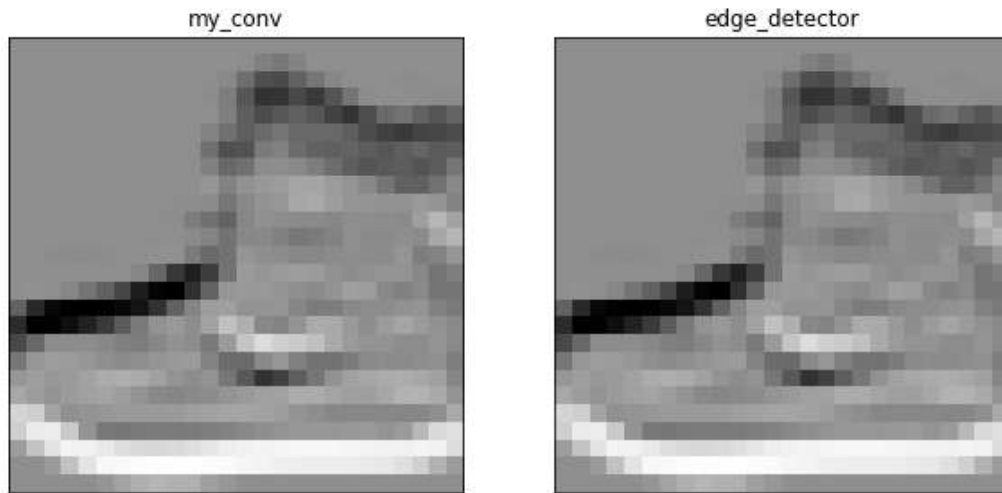
epoch יחיד של אימון מניב את גרעין הקונבולוציה שלהלן, וניתן לראות בו את הדמיון לגרעין מזהה הקצוות.

```
print(my_conv.kern)

Parameter containing:
tensor([[-0.8108, -0.7242, -0.6472],
        [-0.0661,  0.0861, -0.0626],
        [ 0.5860,  1.0309,  0.5985]], requires_grad=True)
```

פלט:

באיור המצורף נראה שהפעלת הפילטרים על אותה תמונה מניבה תוצאה כמעט זהה. אם כן, תהליך הלמידה הסתיים בהצלחה.



כבר בשלב זה אנו יכולים להבחין ביתרונות של שכבות אלו עבור קלט ויזואלי:

- ראשית, מספר הפרמטרים עבור שכבה נתונה קטן במידה ניכרת מאשר בשכבה בעלת קישוריות מלאה – יש ללמוד את הגרעין בלבד.
- שנית, רק מספר מועט של פיקסלים הממוקמים בסמיכות זה לזה בתמונת הקלט משפיעים על חישוב פיקסל של הפלט. כך אנו כופים על הרשת ללמוד כללים המביאים בחשבון אינטראקציה מקומית בלבד, כללים אשר לצופה האנושי ברור שקיימים במידע ויזואלי: קיומו של שרוול חולצה **במקום מסוים בתמונה**, למשל, מעיד על כך שזו תמונה של חולצה.
- נוסף על כך, השימוש החוזר באותו גרעין עבור תתי-מטריצות שונות של תמונת הקלט מוביל את הרשת ללמוד כללים שאינם תלויים במיקום ספציפי במרחב (אין זה משנה היכן בתמונה נמצא השרוול הנ"ל).
- לבסוף, פעולת הקונבולוציה זולה הרבה יותר מחישוב התפשטות לפנים בשכבה לינארית מלאה, שוב עקב כך שכל פיקסל בפלט מחושב על בסיס מעט פעולות חשבון.

שאלות לתרגול

1. תמונות הקלט בצורתן המקורית מכילות ערכים בטווח 0 עד 255. נסו לאמן את שכבת הקונבולוציה ללמוד את גרעין זיהוי הקצוות ללא נרמול לטווח 0 עד 1. אם האימון נכשל, הסבירו מדוע.
2. אסטרטגיית אתחול פרמטרים מוצלחת תביא בחשבון את גודל הגרעין. שנו את בנאי שכבת הקונבולוציה כך שאיברי הגרעין יאותחלו באמצעות דגימה מההתפלגות $U\left[-\frac{1}{\sqrt{K}}, \frac{1}{\sqrt{K}}\right]$ כאשר K הוא מספר האיברים בגרעין. **רמז:** אם $u \sim U[0,1]$ אז $au + b \sim U[b, a + b]$.
3. חשבו באופן אנליטי את ההתפשטות לאחור של השגיאה דרך שכבת קונבולוציה בעלת גרעין בגודל 1×2 , כאשר ידוע שתמונת הקלט היא בגודל 2×3 . כרגיל, הניחו שגרדיאנט המחיר לפי פלט השכבה, $\frac{\partial C}{\partial Y}$, חושב זה מכבר, ועליכם לחשב רק:

- את הגרדיאנט של הפלט לפי הפרמטרים (גרעין הקונבולוציה), $\frac{\partial Y}{\partial K}$, לצורך עדכונים בעת האופטימיזציה,
 - ואת הגרדיאנט לפי קלט השכבה, $\frac{\partial Y}{\partial X}$, לצורך המשך התפשטות השגיאה.
- רמז:** הפרידו למקרים לפי מספר הפעמים שפיקסל מסוים של הקלט משתתף בחישוב פיקסל בפלט.

מבנה שכבת קונבולוציה: ריבוי ערוצים

לשכבת הקונבולוציה אשר כתבנו בפרק הקודם יש להוסיף כמה רכיבים, לפני שנוכל להשתמש בה למשימות למידה מורכבות. בפרק זה נתחיל לסקור רכיבים אלו, ונשלבם בהגדרת השכבה.

ריבוי ערוצי פלט

שכבת הקונבולוציה שלנו בשלב זה תלמד גרעין יחיד בהינתן תמונת הקלט. עם זאת, ייתכן שישנם מאפיינים חשובים אחדים שכדאי לרשת לחלץ מהתמונה לצורך המשך הלמידה (זיהוי קצוות אופקיים, זיהוי קצוות אנכיים, זיהוי פינות וכו'). על כן, נרצה לאפשר לשכבה אחת ללמוד כמה גרעינים שונים באופן בלתי תלוי זה בזה, כאשר כל אחד מהם יחשב קונבולוציה נפרדת של הקלט, והפלט שלה יישמר **בערוץ פלט נפרד**. אם כן, עבור טנזור קלט בגודל $N \times H \times W$ (minibatch בגודל N תמונות מסדר $H \times W$), פלט השכבה יהיה טנזור בממדים $N \times C \times (H - p + 1) \times (W - q + 1)$, כאשר הממד השני יכיל את תוצאות הקונבולוציות של התמונות עם הגרעינים השונים, כולם מסדר $p \times q$. קוד השכבה החדש הוא

```
class ConvLayer_2(nn.Module):
    def __init__(self, out_channels=1, kernel_size=(1,1)):
        super().__init__()
        self.kern = nn.Parameter(
            torch.rand((out_channels,*kernel_size)))
        self.p, self.q = kernel_size
        self.out_channels = out_channels
    def forward(self, X):
        output = torch.empty(X.size(0),
                               self.out_channels,
                               X.size(1)-self.p+1,
                               X.size(2)-self.q+1)
        for i in range(output.size(2)):
            for j in range(output.size(3)):
                sub_img = X[:,i:(i+self.p),j:(j+self.q)]
                sub_img = sub_img.unsqueeze(1)
                output[:, :, i, j] = (sub_img*self.kern).sum(dim=(2,3))
        return output
```

כאן יש לשים לב לכמה פרטים חדשים:

- ראשית, בעת אתחול השכבה יש לציין את מספר ערוצי הפלט המבוקשים. פרט זה הכרחי שכן בעת יצירת אובייקט ממחלקה זו יש לאתחל מספר גרעינים שונים השווה לפרמטר זה.
- שנית, בעת דגימת המ"מ לאתחול גרעיני השכבה, אנו "פותחים" את הזוג הסדור `kernel_size` לשני ערכים נפרדים בעזרת אופרטור ה-* ומייד מאגדים אותם לשלושה סדורה עם הפרמטר `out_channels`. אנו עושים זאת מאחר שגודל הטנזור הנדגם מועבר ל-`torch.rand` בתוך ח"ה סדורה יחידה.
- לבסוף, חישוב כל פיקסל בכל ערוצי הפלט מתבצע בשורת הקוד הקודמת לסוף, וזאת באמצעות שידור תת-התמונה `sub_img` לאורך ממד הערוצים ושידור הגרעינים לאורך ממד ה-`batch`. מכיוון

שלת-התמונה עוד אין ממד המתאים לערוצי הפלט החדשים (גודלה הוא $N \times p \times q$), קודם לשידור אנו מכניסים ממד מנוון לאחר הממד הראשון, בעזרת המתודה `unsqueeze(1)` (גודל תת-התמונה אחרי שימוש במתודה זו הוא $N \times 1 \times p \times q$). ללא הוספת ממד מלאכותי זה נקבל תוצאה שגויה: חוקיות השידור תשווה את ממד הערוץ של הגרעינים (טנזור בגודל $C \times p \times q$) לממד ה-`batch` של תת-התמונה, דבר אשר יוביל לרוב לשגיאה, ובמקרים נדירים לשילוב לא רצוי של רכיבים מממדים שונים.

לצורך הדגמה ניצור שכבת קונבולוציה בעלת שני ערוצי פלט, נגדיר במפורש את הגרעינים שלה ונפעילה על קלט לדוגמה.

```
edge_detector2 = ConvLayer_2(2, (3, 3))
filters = torch.tensor(
    [[[-1., -1, -1],
        [0, 0, 0],
        [1, 1, 1]],

     [[-1, 0, 1],
        [-1, 0, 1],
        [-1, 0, 1]]])
edge_detector2.kern.requires_grad = False
edge_detector2.kern[:] = filters
```

בקטע קוד זה הגדרנו את גרעיני הקונבולוציה של השכבה להיות

$$\text{kern}[0,\dots] = \begin{pmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{pmatrix}, \quad \text{kern}[1,\dots] = \begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix}$$

כך שהערוץ הראשון יזהה קצוות אופקיים והשני קצוות אנכיים.

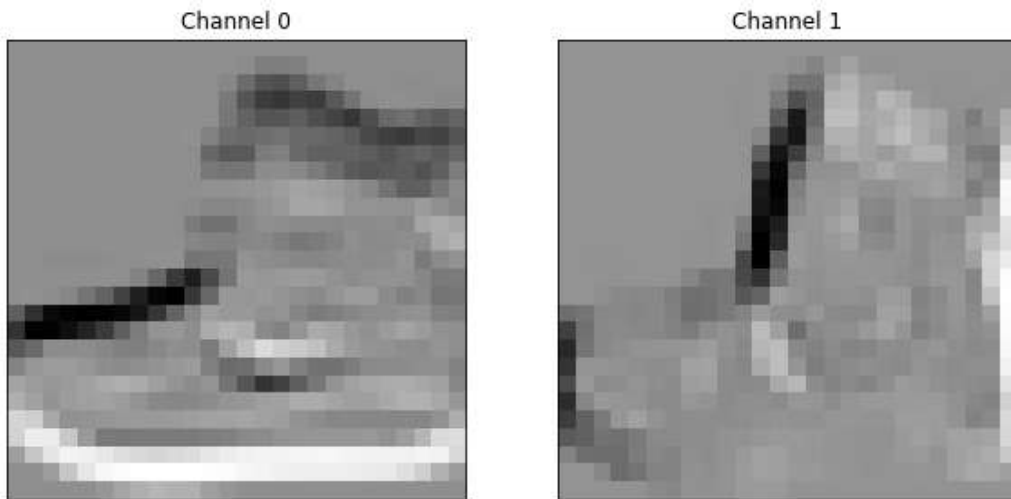
בהנחה שלתוך המשתנה `imgs` טענו כבר `minibatch` של 512 תמונות, הפעלת השכבה עליו תניב את הטנזור שלהלן:

```
edges_detected = edge_detector2(imgs)
print(edges_detected.size())

torch.Size([512, 2, 26, 26])
```

פלט:

בטנזור זה נמצאת התוצאה של שתי הקונבולוציות עבור כל תמונה ב-`minibatch`. באיור המצורף ניתן לראות את שני הערוצים עבור התמונה הראשונה:



ריבוי ערוצי קלט

מטרתנו היא להשתמש בשכבות קונבולוציה ברשת עמוקה, שבה פלט שכבה אחת מועבר לשכבה הבאה בתור. לפיכך, עלינו לשנות את חישוב הקונבולוציה כך שריבוי הערוצים בקלט יובא בחשבון, שכן אף אם קלט הרשת היה תמונה בעלת ערוץ צבע יחיד, בפלט השכבה הראשונה כבר יהיו כמה ערוצים שונים, והחל מהשכבה השנייה לא יהיה די בפעולת הקונבולוציה שהשתמשנו בה עד כה. מובן שהפונקציות שנוסיף לשכבה כעת תאפשר לנו גם להזין לרשת תמונות צבעוניות.

את המעבר לריבוי ערוצי הפלט עשינו באמצעות הגדלת מספר הגרעינים וחישוב קונבולוציות אחדות בריזמנית, אולם שינוי זה אינו מתאים עבור ריבוי ערוצי הקלט. ערוצי הקלט השונים של פיקסל כלשהו בתמונה נתונה מכילים מידע רלוונטי לפיקסל זה וסביבתו: קיומה של פינה באזור מסוים בתמונה יכול לבוא לידי ביטוי בשינויים בשלושת ערוצי הצבע של תמונה צבעונית, למשל. לפיכך, נרצה עתה שפעולת הקונבולוציה תשלב את המידע הקיים בערוצי הקלט השונים, כך שעבור תמונה יחידה בעלת C ערוצים, פלט פעולת הקונבולוציה (עם גרעין נתון) תהיה בעלת ערוץ יחיד.

על כן עלינו לשנות את הגדרתה של הקונבולוציה עצמה, וכן להגדיל את ממד הגרעין: עבור טנזור קלט X בגודל $C \times H \times W$ (המייצג תמונה בעלת C ערוצים באורך H ורוחב W), וגרעין קונבולוציה K בגודל $C \times p \times q$ (כאשר, כבעבר, p ו- q הם פרמטרים קטנים), פלט פעולת הקונבולוציה של X עם K יהיה שוב **מטריצה** Y בגודל $(H - p + 1) \times (W - q + 1)$, בעלת הערכים האלה:

$$y_{r,s} = \sum_{c=1}^C \sum_{i=1}^p \sum_{j=1}^q x_{c,r+i-1,s+j-1} k_{c,i,j}$$

זו הכללה טבעית של הקונבולוציה עם ערוץ יחיד, שכן כדי לחשב את $y_{r,s}$ יש:

1. לחתוך מ- X תת-טנזור בגודל $C \times p \times q$ אשר הפינה השמאלית העליונה שלו בממדי האורך והרוחב היא בעלת האינדקסים r, s .
2. לכפול כל ערוץ של תת-טנזור זה, איבר-איבר, בערוץ המתאים בטנזור הגרעין,
3. לבסוף לסכום את התוצאה.

בקטע הקוד שלהלן נרחיב את הגדרת שכבת הקונבולוציה המקורית, כך שתקבל טנזור קלט בגודל $N \times C \times H \times W$:minibatch של N תמונות בעלות C ערוצים. פלט השכבה יהיה טנזור תלת־ממדי: תוצאת הקונבולוציה עבור כל תמונה ב־minibatch.

```
class ConvLayer_3(nn.Module):
    def __init__(self, in_channels=1, kernel_size=(1,1)):
        super().__init__()
        self.kern = nn.Parameter(
            torch.rand((in_channels,*kernel_size)))
        self.p , self.q = kernel_size
        self.in_channels = in_channels
    def forward(self, X):
        output = torch.empty(X.size(0),
                              X.size(2)-self.p+1,
                              X.size(3)-self.q+1)
        for i in range(output.size(1)):
            for j in range(output.size(2)):
                sub_img = X[:, :, i:(i+self.p), j:(j+self.q)]
                output[:, i, j] = (sub_img*self.kern).sum(dim=(1,2,3))
        return output
```

ראו כי שכבה זו בעלת גרעין קונבולוציה יחיד (תלת־ממדי), וכן שהפחתת הסכום מתבצעת על כל הממדים, מלבד ממד ה־batch.

נמשיך עם הדוגמה הקודמת שבה יצרנו במשתנה edges_detected טנזור בעל שני ערוצים: מזהה קצוות אופקי ומזהה קצוות אנכי. נשלב את המידע מהערוצים השונים בעזרת קונבולוציה עם גרעין 1×1 ושני ערוצי קלט. לאחר הפעלת גרעין זה רוחב ואורך התמונה לא ישתנו, אך בכל פיקסל נקבל צירוף לינארי של ערוצי הקלט. נבחר למשל גרעין המחשב ממוצע של ערכי הערוצים השונים ונקבל את התוצאה הזו.


```

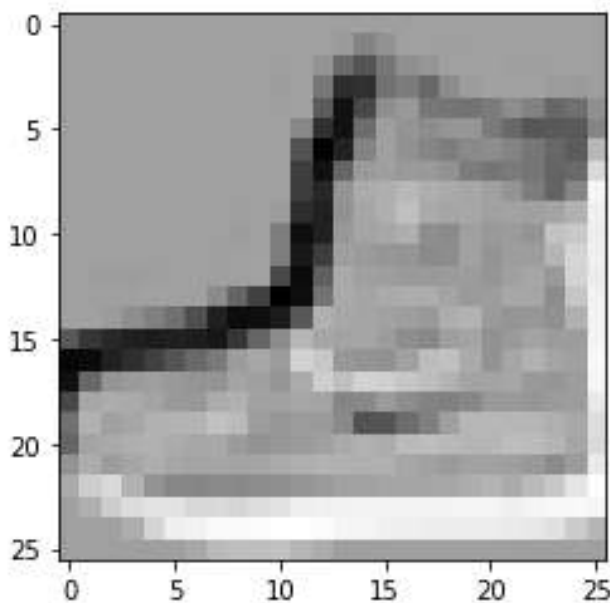
aggregate      = ConvLayer_3(2, (1,1))
with torch.no_grad():
    aggregate.kern[:] = torch.tensor(0.5).expand(2,1,1)
edges_aggregated = aggregate(edges_detected)

print(aggregate.kern.size())
print(edges_aggregated.size())
plt.imshow(edges_aggregated[0,...].detach(), cmap='Greys');

torch.Size([2, 1, 1])
torch.Size([512, 26, 26])

```

פלט:



ראו כיצד פלט השכבה מזהה קצוות אופקיים או אנכיים.

שאלה לתרגול

כתבו שכבת קונבולוציה בעלת מספר ערוצי קלט ופלט: הקלט לשכבה יהיה טנזור בגודל $N \times C_{in} \times H \times W$ והפלט – טנזור בגודל $N \times C_{out} \times (H - p + 1) \times (W - q + 1)$. השכבה תחשב C_{out} קונבולוציות שונות (עם גרעינים שונים) על כל אחת מ- N תמונות הקלט.

חתימת בנאי השכבה תהיה

```
__init__(self, in_channels=1, out_channels=1, kernel_size=(1,1))
```

רכיבים נוספים של שכבות קונבולוציה

לאחר שהוספנו ערוצי פלט וקלט מרובים לשכבות הקונבולוציה בפרק הקודם, נסקור כעת את הרכיבים האחרונים הדרושים כדי להשתמש בהן במשימות למידה.

ריפוד (padding)

בפרקים הקודמים פגשנו בתכונה יסודית של פעולת הקונבולוציה: גודל תמונת הפלט קטן מתמונת הקלט בממדי האורך והרוחב, וזאת בהתאם לגודל גרעין הקונבולוציה. לעתים הקטנת ממד זו אינה רצויה, וכדי לשלוט על גודל הפלט אפשר להגדיל באופן מלאכותי את הקלט של השכבה באמצעות הוספת שורות ועמודות בקצות התמונה. שיטה זו נקראת ריפוד (padding) וישנם כמה מימושים שלה ב־PyTorch, בהתאם לערכים המוצבים בעמודות או בשורות החדשות. הבחירה הנפוצה ביותר היא ריפוד באפסים. ראו בדוגמה שלהלן כיצד אנו מרפדים טנזור קלט בשורת אפסים אחת ובעמודות אפסים אחת נוספת מכל צד.

```
pad_layer = nn.ZeroPad2d(1)
A = torch.arange(1,10).reshape(3,3)
print(A, pad_layer(A), sep='\n')
```

פלט:

```
tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])
tensor([[0, 0, 0, 0, 0],
        [0, 1, 2, 3, 0],
        [0, 4, 5, 6, 0],
        [0, 7, 8, 9, 0],
        [0, 0, 0, 0, 0]])
```

טרנספורמציה זו ממומשת גם בתוך שכבת הקונבולוציה הסטנדרטית של PyTorch: בשורת הקוד שלהלן אנו מגדירים שכבת קונבולוציה המקבלת minibatch של תמונות קלט בעלות ערוץ מידע יחיד, טנזור בגודל $N \times 1 \times H \times W$, ומחזירה פלט באותו גודל בדיוק. בהתאם לגודל הגרעין, השכבה בוחרת אוטומטית את מספר השורות והעמודות של הריפוד.

```
conv_layer = nn.Conv2d(in_channels=1, out_channels=1,
                        kernel_size=(3, 5), padding="same")
```

פרמטר bias

לשכבת הקונבולוציה המובנית ב־PyTorch שני רכיבים נוספים שעוד לא דנו בהם. בראשון נפגוש תוך כדי התבוננות ברשימת הפרמטרים בשכבה שזה עתה הגדרנו.

```
for params in conv_layer.named_parameters():
    print(params)
```

פלט:

```
('weight', Parameter containing:
tensor([[[[-0.2093, -0.2208,  0.0425,  0.1462, -0.1257],
          [ 0.0535, -0.0787, -0.0518,  0.1329,  0.2383],
          [ 0.0557,  0.1856, -0.1075, -0.2001,  0.1463]]]],
requires_grad=True))
('bias', Parameter containing:
tensor([-0.2201], requires_grad=True))
```

ראו שהפרמטר `weight` מכיל את גרעין הקונבולוציה בגודל 3×5 (עם שני ממדים מנוונים נוספים – עבור ערוצי הקלט והפלט), כצפוי. נוסף על כך, יש לשכבה פרמטר בשם `bias`, המכיל ערך נפרד לכל ערוץ פלט (אחד במקרה שלנו), ומוסיפים אותו לתוצאת הקונבולוציה בערוץ זה. הצורך בפרמטר זה נובע ממבנה הרשת שבה שכבת הקונבולוציה תמצא שימוש: לרוב, פלט השכבה יעבור מיידית לאקטיבציה לא לינארית, כגון ReLU, שעבורה דרוש ה-`bias` כדי לווסת את הסף שבו הנוירון מופעל. אין אנו מחברים שכבות קונבולוציה זו לזו ללא אקטיבציה לא לינארית ביניהן, שהיא הקונבולוציה היא פעולה לינארית בבסיסה, ועל כן שרשרת קונבולוציות זו לאחר זו שקול לביצוע פעולה לינארית יחידה.

גודל פסיעה (stride)

הרכיב הנוסף השני של שכבות קונבולוציה שנדון בו כעת הוא פרמטר גודל הפסיעה. השתמשנו בריפוד הקלט למטרת הגדלת הפלט, אולם לעתים נרצה דווקא את התוצאה ההפוכה – לצמצם את גודל התמונה לאחר המעבר בשכבה, למשל כאשר גודל הקלט גדול מדי לאימון הרשת ביעילות, או כאשר מעוניינים לקבל לאחר מספר מועט של שכבות מאפיינים היוצרים קשר בין אזורים מרוחקים בתמונת הקלט.

בעבר עברנו על **כל פיקסל** בקלט (אשר מימינו ומתחתיו ישנם די איברים, כגודל הגרעין) וחתכנו מהקלט מטריצה בגודל הגרעין לטובת חישוב פיקסל יחיד בפלט, ואילו כעת נעשה זאת רק עבור חלק מהפיקסלים לפי חוקיות פשוטה, שנבין מהדוגמה שלהלן:

```

downsample = nn.Conv2d(in_channels=1, out_channels=1,
                        bias=False, kernel_size=(1, 1),
                        stride=(2, 3))
with torch.no_grad():
    downsample.weight[:] = torch.tensor(1.).reshape(1, 1, 1, 1))
A = torch.arange(25, dtype=torch.float).reshape(1, 1, 5, 5)
print(A, downsample(A), sep='\n')

```

פלט:

```

tensor([[[[ 0.,  1.,  2.,  3.,  4.],
           [ 5.,  6.,  7.,  8.,  9.],
           [10., 11., 12., 13., 14.],
           [15., 16., 17., 18., 19.],
           [20., 21., 22., 23., 24.]]]]])
tensor([[[[ 0.,  3.],
           [10., 13.],
           [20., 23.]]]], grad_fn=<ThnnConv2DBackward0>)

```

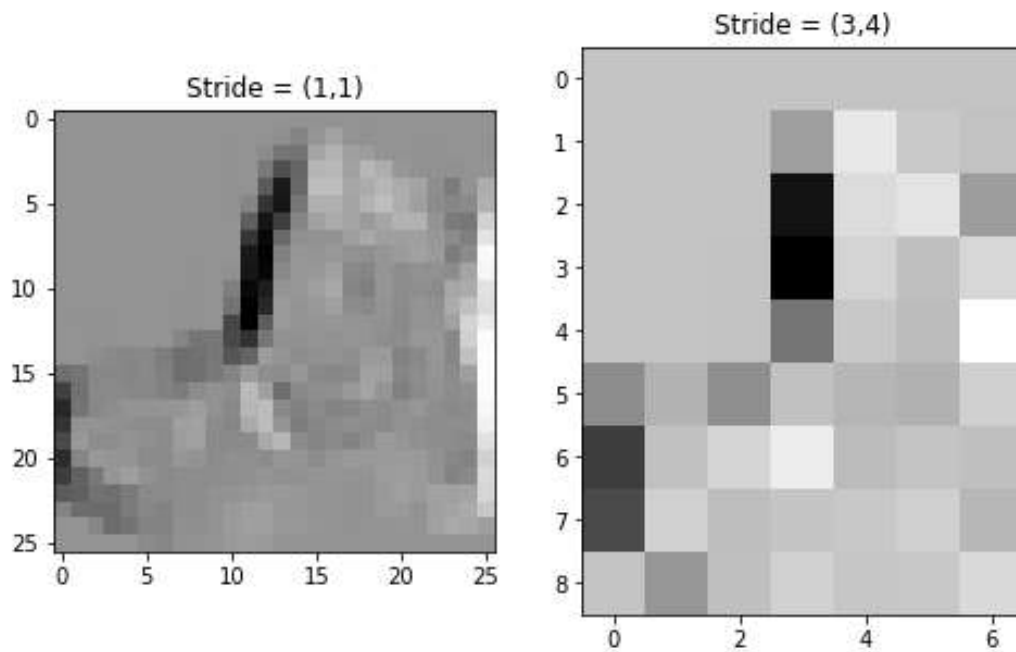
בדוגמה זו הגדרנו שכבת קונבולוציה ללא bias עם גרעין בגודל 1X1, ואת ערך הגרעין קבענו להיות 1. קונבולוציה עם גרעין כזה, עבור קלט בעל ערוץ יחיד, אמורה להניב את פונקציית הזהות, אך שימו לב שהעברנו לבנאי השכבה גם פרמטר stride, גודל הפסיעה. משמעות הזוג הסדור שהועבר בפרמטר זה היא שבעת חישוב הקונבולוציה יש לחשב ולשמור את ערך מכפלת תת-המטריצה בגרעין עבור אחד מכל שלושה איברים בשורה (כל עוד ניתן לבצע חישוב זה, כאמור), ואחת מכל שתי שורות. התוצאה המתקבלת עבור הגרעין המסוים שבחרנו היא שרק האיברים מהעמודה הראשונה או הרביעית הנמצאים בשורות האיזוגיות נשמרים בפלט.

מובן שניתן לשלב את פרמטר הפסיעה עם כל אחד מהרכיבים האחרים של שכבת הקונבולוציה. למשל, נגדיר מחדש את מזהה הקצוות האופקיים ששימש כדוגמה בפרק הקודם עם פסיעות של שורה אחת מתוך כל שלוש ועמודה אחת מתוך כל ארבע בקטע הקוד שלהלן, ונשווה את תוצאות הפילטר החדש למקורי באיור העוקב.

```

edge_detector2 = nn.Conv2d(in_channels=1, out_channels=1,
                            bias=False, kernel_size=(3, 3),
                            stride=(3, 4))
with torch.no_grad():
    edge_detector1.weight[:] = torch.tensor(
        [[[-1., 0, 1], [-1, 0, 1], [-1, 0, 1]]]).reshape(1, 1, 3, 3)

```



איגום (pooling)

שיטה נוספת לצמצום ממד הפלט היא העברת הפלט של שכבת הקונבולוציה (לאחר אקטיבציה) לשכבת איגום (pooling layer). שכבה זו תשקלל את הנתונים של כמה פיקסלים סמוכים זה לזה, למשל באמצעות חישוב הממוצע שלהם, ליצירת מאפיין יחיד בפלט. כתוצאה יתקבלו מאפיינים אשר אינם רגישים לתזוזות קטנות של הקלט, דבר הרצוי לרוב: מעניין אותנו אם קיימים בתמונת הקלט שרוכי נעליים או לא, ולא דווקא המקום המדויק שלהם.

ראו בדוגמה המצורפת כיצד אנו מגדירים שכבת איגום אשר מחלקת את הקלט לתת־מטריצות זרות מסדר 2×3 ומחזירה מאפיין יחיד לכל תת־מטריצה – הערך המרבי בה.

```
pool = nn.MaxPool2d(kernel_size=(2,3))
A = torch.arange(40, dtype=torch.float).reshape(1,1,5,8)
print(A,pool(A),sep='\n')
```

פלט:

```
tensor([[[[ 0.,  1.,  2.,  3.,  4.,  5.,  6.,  7.],
           [ 8.,  9., 10., 11., 12., 13., 14., 15.],
           [16., 17., 18., 19., 20., 21., 22., 23.],
           [24., 25., 26., 27., 28., 29., 30., 31.],
           [32., 33., 34., 35., 36., 37., 38., 39.]]]])
tensor([[[[10., 13.],
          [26., 29.]]]])
```

בדוגמה זו שתי העמודות האחרונות והשורה האחרונה בקלט "אבדו" – הן לא השתתפו בתהליך האיגום, שכן ממדי הקלט לא תואמים בדיוק את גודל המטריצה המשמשת לאיגום. כדי להימנע מאובדן זה ניתן להעביר לשכבה פרמטר הדורש להשתמש בכל המאפיינים בקלט, במחיר של מטריצת

איגום קטנה יותר בקצוות. למשל בקטע הקוד שלהלן, המאפיין השלישי בשורה הראשונה בפלט מחושב כערך המרבי של מטריצה מסדר 2×2 .

```
pool = nn.MaxPool2d(kernel_size=(2,3), ceil_mode=True)
print(A, pool(A), sep='\n')
```

פלט:

```
tensor([[[[ 0.,  1.,  2.,  3.,  4.,  5.,  6.,  7.],
           [ 8.,  9., 10., 11., 12., 13., 14., 15.],
           [16., 17., 18., 19., 20., 21., 22., 23.],
           [24., 25., 26., 27., 28., 29., 30., 31.],
           [32., 33., 34., 35., 36., 37., 38., 39.]]]])
tensor([[[[10., 13., 15.],
           [26., 29., 31.],
           [34., 37., 39.]]]])
```

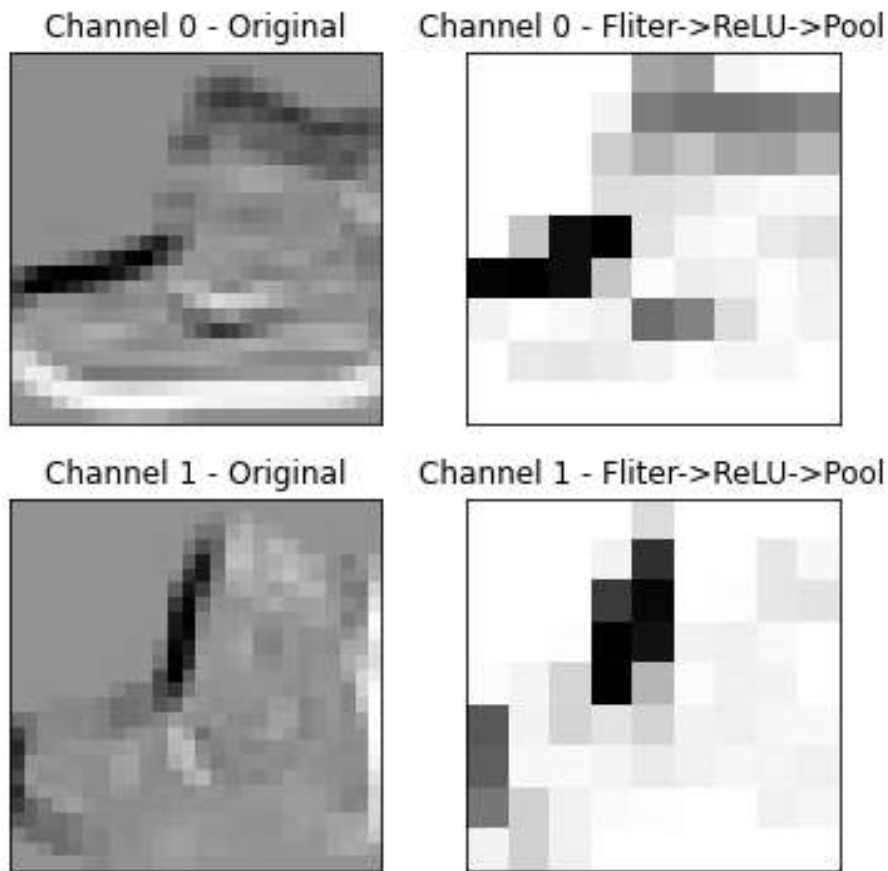
בדומה לשכבת קונבולוציה, גם לשכבת איגום יש פרמטר `stride`, המאפשר לקבוע ברזולוציה גבוהה את החפיפה בין תתי-מטריצות אשר ישמשו ליצירת מאפייני הפלט. ברירת המחדל משתמשת במטריצות זרות, כאילו הועבר הפרמטר `stride=kernel_size`. לבסוף, נציין כי עבור קלט רב-ערוצי שכבת האיגום מחשבת פלט לכל ערוץ בנפרד.

לסיכום, ראו בקטע הקוד המצורף כיצד אנו מגדירים בלוק של שלוש שכבות, המשרשר את פעולת הקונבולוציה, האקטיבציה והאיגום. זהו הרצף שנשתמש בו לרוב בתוך רשת נוירונים, בציפייה שהרשת תלמד בתהליך האימון את גרעין הקונבולוציה ואת פרמטר ה-`bias`. בדוגמה זו אנו מגדירים דינתי את הגרעין ואת מזהה הקצוות האופקיים והאנכיים שהשתמשו בו בעבר.

```
edge_detector2 = nn.Conv2d(in_channels=1, out_channels=2,
                           bias=False, kernel_size=(3, 3),
                           stride=(1,1))
with torch.no_grad():
    edge_detector2.weight[:] = torch.tensor(
        [[[-1., -1, -1],
           [0, 0, 0],
           [1, 1, 1]],
         [[-1, 0, 1],
           [-1, 0, 1],
           [-1, 0, 1]]]).reshape(2, 1, 3, 3))

block = nn.Sequential(edge_detector2,
                      nn.ReLU(),
                      nn.MaxPool2d(kernel_size=(3,3),
                                   ceil_mode=True))
```

פלט הבלוק (בגודל 9×9 פיקסלים) עבור תמונה אחת מאויר להלן, לצד הפלט המקורי של מזהה הקצוות לבדו.



שאלות לתרגול

- עבור גרעין קונבולוציה בגודל $p \times q$, בכמה שורות ועמודות ריפוד יש להשתמש כדי לשמור על ממדיה המקוריים של תמונת הקלט לאחר הפעלת הקונבולוציה?
- א. הגדירו שכבת קונבולוציה עם גרעין בגודל 7×5 ללא פרמטר bias. בשכבה יהיה ערוץ קלט יחיד וערוץ פלט יחיד.
ב. בחרו תמונת קלט אקראית מהאוסף Fashion-MNIST והעבירו אותה דרך השכבה.
ג. רפדו את תמונת הקלט בשורות ובעמודות אפסים מלמעלה ומשמאל בלבד, כך שהפלט יהיה מאותו ממד כמו הקלט. העבירו את התוצאה דרך שכבת הקונבולוציה והדפיסו או איירו את התוצאה.
ד. האם תוכלו לשער מדוע נהוג למקם את האפסים הנוספים מסביב לתמונת המקור, ולא רק בצד אחד שלה?
- הציגו את הקונבולוציה של תמונת קלט X בגודל $1 \times 5 \times 5$ (בעלת ערוץ יחיד) עם גרעין בגודל 2×2 כהעתקה לינארית.

הנחיות:

- ראשית מצאו את הממד של פלט הקונבולוציה.
- שטחו את תמונת הקלט והציגו אותה כווקטור $\hat{X}$ בגודל $HW \times 1$.
- כעת מצאו מטריצה A כך ש- $A\hat{X}$ היא תוצאת הקונבולוציה, אף היא לאחר שיטוח.
- מצאו ראשית את ממדי A בעזרת ההנחיות הקודמות.

o רוב ערכיה של A יהיו אפסים.

4. העבירו לבנאי של `nn.Conv2d` בריזמנית פרמטר גודל פסיעה גדול מ-1 ואת הפרמטר `padding="same"`. הסבירו מדוע מתקבלת שגיאה.

5. צרו שכבת קונבולוציה בעזרת המחלקה `nn.Conv2d` והגדירו את הגרעין שלה ידנית, כך שהיא תבצע פעולה זהה לשכבת איגום הנוצרת כך:

```
pool = nn.AvgPool2d(kernel_size=(2,2))
```

הערה: שימו לב ששכבת איגום זו מחשבת את הממוצע של כל תת־מטריצה מסדר 2×2 .

רשתות קונבולוציה מודרניות

רשתות קונבולוציה נמצאות בשימוש החל משנות ה־90 של המאה ה־20: אחת הרשתות הנפוצות המוקדמות היא LeNet-5, אשר אומנה לזהות ספרות בכתב יד מתמונות בגודל 28X28 בעלות ערוץ צבע יחיד – תמונות בעלות ממדים זהים לאלו של אוסף הנתונים MNIST. רשת זו הראתה ביצועים שווים ערך לאלגוריתמי זיהוי תמונה אלטרנטיביים המשתמשים בחילוץ מאפיינים מהונדס מראש למטרה הנדרשת והזנת מאפיינים אלו לאלגוריתם למידה. זאת כאשר הקלט לרשת היה תמונות מקוריות, ללא כל עיבוד מקדים.

עם זאת, הגידול העצום בפופולריות של רשתות קונבולוציה, ושל למידה עמוקה בכלל, התרחש עקב הביצועים יוצאי הדופן של הרשת AlexNet בתחרות ILSVRC-2012: תחרות זיהוי תמונה המבוססת על אוסף הנתונים הענק ImageNet. בפרק הנוכחי נסקור את הרכיבים אשר הובילו לניצחונה בתחרות, רכיבים אשר הפכו סטנדרטיים בתכנון המבנה ותהליך האימון של רשתות נוירונים עמוקות כיום.

אוסף הנתונים ImageNet

בשנת 2009 פורסם לראשונה מסד הנתונים ImageNet ובו מיליוני תמונות שנאספו ממנועי חיפוש וממאגרי תמונות מקוונים, כגון Flickr. תמונות אלו סווגו בידי משתמשים אנושיים למעל 20,000 קטגוריות שונות, חלקן חופפות (למשל כלב באופן כללי וכלב מגזע מסוים), ולפיכך תמונה נתונה סווגה לעתים ברזמנית לקטגוריות אחדות.

מתוך מסד זה בחרו מארגני התחרות ILSVRC אלף קטגוריות **זרות** זו לזו, וסיפקו למשתתפי התחרות כ־1,000 דגימות מכל קטגוריה, ובסך הכול כ־1.2 מיליון תמונות לאימון. תחרויות פופולריות קודמות השתמשו בסט נתונים של עשרות אלפי תמונות המסווגות לעשרות אחדות של קטגוריות, ולכן זמינותו של סט נתונים בסדר גודל שכזה הייתה קרקע פורייה להצלחתן של רשתות נוירונים, אשר כזכור מצטיינות כאשר סט נתוני האימון גדול במיוחד.

גודל סט נתונים זה אינו מאפשר סיווג אנושי מדויק ומלא של כל התמונות: ייתכן למשל שקיימת באוסף נתונים תמונה השייכת לשתי קטגוריות ברזמנית, אך סווגה רק לאחת מהן. כדי להתמודד עם אידיוקים כאלו, המדד להצלחה בתחרות נבחר להיות דיוק של 1 מתוך 5 (top-5 accuracy), שבו האלגוריתם הנמדד הציע חמש קטגוריות אפשריות עבור כל תמונה מסט הבדיקה. אם הקטגוריה הנכונה היא אחת מחמש אלו, נזקפה לזכות האלגוריתם הצלחה בסיווג התמונה הנתונה. במדד זה AlexNet הגיעה לשיעור סיווג נכון של 84.7%, בעוד האלגוריתם הבא בתור בתחרות זו, אשר השתמש בגישה קלאסית של חילוץ מאפיינים ומסווגים לינאריים, הגיע לשיעור סיווג של 73.8% בלבד. בשנת 2015 אחוז ההצלחה של האלגוריתם המנצח (גם הוא רשת קונבולוציה, ונלמד עליה בפרק הבא) כבר היה מעל ל־95%.

עיבוד מקדים והעשרת אוסף הנתונים

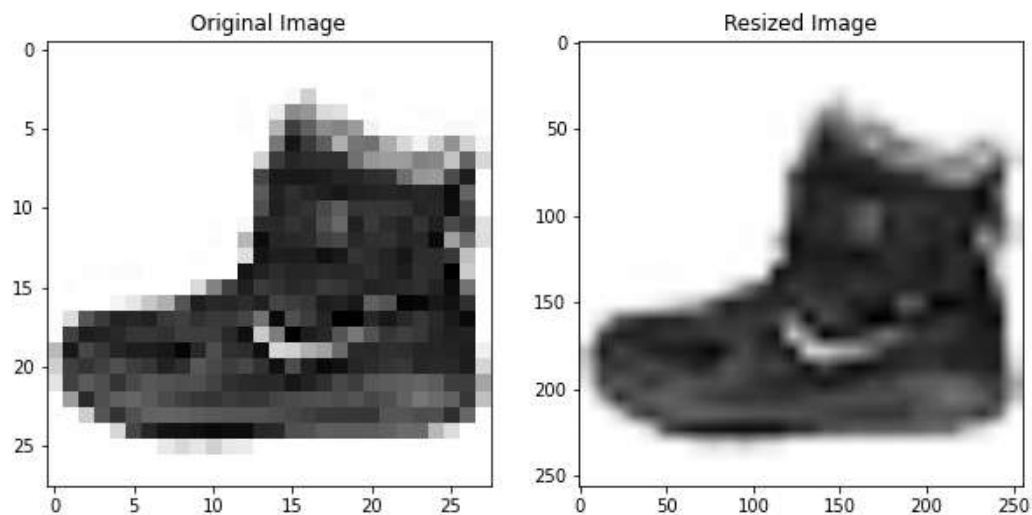
התמונות ב־ImageNet נאספו מרחבי האינטרנט ולכן לרוב הרזולוציה שלהן גבוהה מדי כדי לשמש כקלט אפילו עבור רשת קונבולוציה. לפיכך, לפני הזנתן לרשת הן הוקטנו לגודל 256X256. בתהליך האימון תוצאה זו לא הוזנה ישירות אל הרשת אלא שימשה מקור להעשרת אוסף הנתונים: בעזרת טרנספורמציות פשוטות, כגון שיקופים, סיבובים, שינויי צבע קלים וכו', ניתן ליצור מתמונה מקורית

אחת אלפי תמונות נוספות מבלי לשנות את הקטגוריה שהיא שייכת אליה, ובכך להגדיל את סט הנתונים באופן מלאכותי ולהימנע מהתאמת יותר.

עבור נותנים ויזואליים קיימת ב־PyTorch חבילת פונקציות נרחבת לעיבוד מקדים זה, ולפני שנמשיך את הדיון ב־AlexNet, נדגים את השימוש שלה להלן.

```
import torchvision.transforms as T
resize = T.Resize(256)
new_img = resize(img)
```

לאחר הרצת קטע הקוד הנ"ל, נקבל במשתנה `new_img` תמונה אשר הוגדלה או הוקטנה ל־ 256×256 . ראו באיור שלהלן את השפעת הפונקציה על תמונה מהאוסף Fashion-MNIST.



לרוב נרצה להשתמש בכמה פונקציות של עיבוד מקדים, ולצורך זה ניתן לשרשרן כדלהלן.

```
transforms = T.Compose([T.Resize(256),
                        T.RandomCrop(224),
                        T.RandomHorizontalFlip(0.5),
                        T.RandomRotation(30),
                        T.RandomPerspective(distortion_scale=0.3, p=0.5)])
new_img = transforms(img)
```

להלן מאוירות תוצאות אחדות של הפעלת רצף טרנספורמציות זה, אשר מגדיל, חותך, משקף סביב האמצע, ומסובב בדרכים שונות ובאקראי את תמונת הקלט:



בכל התמונות ניכר כי זהו מגף ושימוש בסט אימון המועשר בתמונות אלו יניב רשת אשר למדה לזהות מגף גם דרך טרנספורמציות אלו, ובהכרח מותאמת פחות לסט הנתונים המקורי.

לבסוף, ניתן לשלב טרנספורמציות אלו ישירות בתהליך טעינת הנתונים לזיכרון, באמצעות הגדרתן בפרמטר `transform` של בנאי אובייקט ה-`Dataset`. בצורה זו, הטרנספורמציות יופעלו פעם אחת על כל תמונה בעת טעינת כל `minibatch`, וכך הרשת תיחשף לתמונות "חדשות" בכל `epoch`.

נקודה עדינה שדורשת תשומת לב כאשר אנו מעשירים את סט הנתונים היא שאת סט הבדיקה יש להעביר אל הרשת דרך טרנספורמציות דומות, כדי להתאימו לקלט שהרשת אומנה עליו. עם זאת, ברצוננו להימנע מהפעלת טרנספורמציות אקראיות על סט הבדיקה, כדי שהחיזוי לא יהיה תלוי בהגרלת המשתנים המקריים. אם כן, סדרת הטרנספורמציות עבור סט הבדיקה המתאימה לסט אימון שהועשר כנ"ל מופיעה בקטע הקוד שלהלן, ואותה יש להעביר להגדרת אובייקט ה-`Dataset` של סט הבדיקה. ראו כי הגזירה האקראית הוחלפה בערך הממוצע שלה: גזירה במרכז התמונה.

```
test_transforms = T.Compose([T.ToTensor(),
                             T.Resize(256),
                             T.CenterCrop(224)])
```

ארכיטקטורת הרשת AlexNet

בקטע הקוד המצורף מופיעה ארכיטקטורת הרשת AlexNet, כמחלקה של PyTorch. שימו לב שזו הפעם הראשונה שבה אנו מגדירים רשת שלמה כמחלקה היורשת מ-`nn.Module`, אך אין זה שונה מהגדרת שכבה יחידה בצורה זו.

```

class AlexNet(nn.Module):
    def __init__(self, dropout=0.5):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, 11, stride=4, padding=2),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(64, 192, 5, padding=2),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(192, 384, 3, padding=1),
            nn.ReLU(),
            nn.Conv2d(384, 256, 3, padding=1),
            nn.ReLU(),
            nn.Conv2d(256, 256, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=3, stride=2))
        self.classifier = nn.Sequential(
            nn.Dropout(p=dropout),
            nn.Linear(256 * 6 * 6, 4096),
            nn.ReLU(),
            nn.Dropout(p=dropout),
            nn.Linear(4096, 4096),
            nn.ReLU(),
            nn.Linear(4096, 1000))

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x.flatten(start_dim=1))
        return x

```

מעיון בארכיטקטורה עולים כמה פרטים מעניינים. ראשית, נשים לב שפלט הרשת הוא שכבה לינארית – אלו הציונים הגולמיים אשר יש להעביר לפונקציית softmax ולקבל את ההסתברויות השייכות לכל אחת מ־1,000 המחלקות. כזכור, בעזרת הסתברויות אלו אנו מחשבים את פונקציית המחיר המתאימה למשימת סיווג, היא האנטרופיה הצולבת. ב־PyTorch ניתן להעביר ציונים אלו ישירות לפונקציית המחיר המבצעת חישוב זה ביעילות. על כן, כדי לאמן רשת זו יש להגדיר את פונקציית המחיר כך:

```
CE_loss = nn.CrossEntropyLoss()
```

ולא בעזרת הפונקציה `nn.NLLLoss`, כפי שעשינו בעבר, כאשר בסוף הרשת הייתה שכבת `.nn.LogSoftmax`.

שנית, גרעין הקונבולוציה הגדול בשכבה הראשונה והשימוש בשכבות איגום מפחיתים את גודל הקלט ל־ 13×13 באמצע הרשת, ואף ל־ 6×6 לאחר שכבת האיגום האחרונה. עם זאת, מספר הערוצים גדל עד לערך המרבי 384. הרעיון הניצב מאחורי טרנספורמציות אלו הוא לאפשר לרשת ללמוד מספר רב של

מחלצי מאפיינים המתייחסים לחלקים נרחבים ברשת, שהרי עקב הפחתת הממד, פיקסל יחיד במאפיין מאמצע הרשת מחושב על בסיס מספר רב של פיקסלים בתמונה המקורית.

פרט שלישי מעניין הוא השימוש בשכבות dropout ובפונקציית האקטיבציה ReLU, שאמנם שגורות בשימוש המודרני, אך היו בתחילת דרכן בעת פרסום המאמר, ואין ספק שהצלחתה של AlexNet תרמה לפופולריות של רכיבים אלו.

לבסוף, ניתן לראות שהחישוב ברשת מחולק לשניים: רשת קונבולוציה המחלצת מאפיינים ומעבירה אותם למסווג סטנדרטי בעל קישוריות מלאה – זהו "ראש הסיווג" אשר ניתן להחלפה אם ברצוננו לבצע משימה אחרת, כגון רגרסיה, על בסיס רשת זו.

שני פרטים חשובים אשר לא באים לידי ביטוי במימוש הנ"ל, אך היו חלק בלתי נפרד מהצלחתה הם השימוש בנרמול, בדומה ל-batch normalization, על בסיס ממד הערוץ במקום ממד ה-batch, וכן אימון הרשת על גבי מאיץ גרפי. אפילו תוך כדי שימוש בשני מאיצים בריזמנית – נדרש זמן ריצה של כשבוע עבור אימון הרשת המקורית.

שאלות לתרגול

- א. הגדירו מחלקה היורשת מ-`nn.Module` ובה ממומשת ארכיטקטורת הרשת LeNet-5 (מצאו איור של ארכיטקטורה זו במאמר המקורי הנמצא באתר הקורס). ניתן להחליף רכיבים כגון פונקציית האקטיבציה `tanh` בחלופות מודרניות, כגון ReLU.
- ב. אמנו את הרשת לזהות את פריטי הלבוש מאוסף הנתונים Fashion-MNIST, עד אשר מתקבלת התאמת יתר ברורה.
- ג. לפני הזנת התמונות ברשת בצעו טרנספורמציות שונות על תמונות הקלט למטרת העשרת אוסף הנתונים ואמנו את הרשת שוב. האם התקבלה התאמת יתר?
- העבירו דרך הרשת AlexNet דגימה אקראית של קלט בממדים הצפויים (תמונות צבע $3 \times 224 \times 224$), תוך כדי הדפסת גודל הטנזור לאחר מעבר בכל שכבה.
- רמז: השתמשו במתודה `module.children()` המחזירה גנרטור של תת-השכבות במודול.
- מהו מספר הפרמטרים ברשת AlexNet?

הנחיות:

- מצאו את גודל גרעין הקונבולוציה עבור כל שכבה, תוך כדי התחשבות במספר ערוצי הקלט והפלט. זכרו שלכל ערוץ פלט יש גם פרמטר `bias`.
- זכרו שבשכבה לינארית כל נוירון פלט מחובר לכל נוירון קלט, ולכל חיבור כזה קיים פרמטר.
- תוכלו לעשות זאת אוטומטית באמצעות מעבר על כל השכבות ברשת וצבירת הגודל של הפרמטרים שלהן.
- על סמך התשובה לשאלה הקודמת, שערו מדוע בארכיטקטורת AlexNet השתמשו בשכבות dropout במסווג בלבד.

רשתות שיוניות

בפרק זה נסקור את מבנה הרשת ResNet, אשר שימשה כבסיס לאלגוריתם המנצח בתחרות ImageNet בשנת 2015, בשיעור דיוק top 5 accuracy של 96.43%. הסוד להצלחתה של ResNet הוא שימוש ברשתות עמוקות במיוחד: בעוד הרשתות הזוכות בתחרות בשנים הקודמות היו בעלות 20–30 שכבות, מפתחי ResNet השתמשו ברשתות בעלות עד 150 שכבות.

באותן שנים היה ידוע בקהילת זיהוי התמונה שרשתות קונבולוציה עמוקות מצליחות להכליל טוב יותר משטוחות: המבנה ההיררכי של השכבות מאפשר לרשת לחלץ מאפיינים מורכבים מתמונת המקור, מאפיינים השימושיים למשימת הסיווג העוקבת. עם זאת, רשתות עמוקות מציבות אתגר קשה יותר לאימון, עקב הצורך לחשב את הגרדיאנט דרך מספר רב של שכבות. חישוב גרדיאנט זה עשוי להיות בעייתי, שכן התפשטות הגרדיאנט לאחור דרך שכבה נתונה מבוצעת באמצעות **כפל** הגרדיאנט הקודם בגרדיאנט של שכבה זו. תוצאת הכפל של מספר רב של ערכים נוטה להתאפס (אם כופלים לרוב ערכים קטנים), או לשאוף לאינסוף (אם כופלים לרוב ערכים גדולים) – זוהי התופעה של הגרדיאנט המתאפס או המתפוצץ, אשר מובילה לאייתכנסות אלגוריתם האופטימיזציה.

הוספת שכבות נורמליזציה כגון Batch-Norm ואתחול נכון של ערכי הפרמטר הפכו את תהליך האימון של רשתות קונבולוציה עמוקות, עם עשרות שכבות, לאפשרי, אך רשתות עמוקות יותר עדיין הפגינו ביצועים פחותים, הן על סט הבדיקה כמובן והן על סט האימון עצמו, כך שלא היה מדובר בהתאמת יתר. דבר זה עומד בסתירה לאינטואיציה, שכן על פניו, רשת עמוקה יכולה ללמוד לחלץ מאפיינים זהים לאלו של רשת בעלת פחות שכבות ולהעביר אותם הלאה מבלי לבצע כל חישוב נוסף בשכבות הנוספות, וכך להגיע לביצועים זהים.

הבחנות אלו הובילו את צוות החוקרים האחראי לפיתוח ResNet להסיק כי פתרון אפשרי לבעיה יהיה לאפשר לשכבות ברשת ללמוד את פונקציית הזהות בצורה קלה יותר. פתרון זה בא לידי ביטוי ברכיב פשוט אשר נתאר בפירוט מיידי, המכונה חיבור הדילוג (skip connection). חיבור כזה עוקף כמה שכבות, ונועד להפוך את פונקציית הזהות ל"ברירת המחדל" של שכבות אלו. תוך כדי שימוש ברכיב זה, על השכבות המוקפות בחיבור דילוג ללמוד רק את ה**התוספת** לפונקציית הזהות שיש לחלץ, ואם אין כזו – איפוס הפרמטרים של השכבות יוביל להתפשטות לפנים של פונקציית הזהות.

בלוקים שיוניים

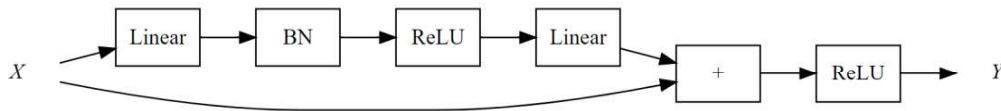
חיבורי דילוג באים לידי ביטוי בבלוקים שיוניים (residual blocks), אשר הרשת ResNet מורכבת משרשר שלם ברצף. תחילה נגדיר בלוק שיוני עבור רשת בעלת קישוריות מלאה, כדי לדון בתכונותיו, ואחרי כן נעבור להגדרת הבלוק המקורי, עבור רשתות קונבולוציה.

```

class ResBlockMLP(nn.Module):
    def __init__(self, in_features):
        super().__init__()
        self.relu = nn.ReLU()
        self.Z1 = nn.Linear(in_features, in_features)
        self.bn = nn.BatchNorm1d(in_features)
        self.Z2 = nn.Linear(in_features, in_features)
    def forward(self, X):
        Y1 = self.Z1(X)
        Y1 = self.bn(Y1)
        Y1 = self.relu(Y1)
        Y1 = self.Z2(Y1)
        Y2 = Y1+X          #skip connection
        Y = self.relu(Y2)
        return Y

```

ראו בהגדרת השכבות הלינאריות בעת אתחול הבלוק שמספר ניוירני הפלט (הפרמטר השני המועבר לבנאי השכבה `nn.Linear`) שווה למספר ניוירני הקלט. בחירה זו אינה אקראית, שכן בהתפשטות לפנים דרך הבלוק יש לסכום את פלט שכבות אלו, איבראיבר, עם הקלט המקורי, ועל כן הממדים חייבים להיות זהים. חישוב זה, הנשמר במשתנה $Y2$ הופך בלוק זה לשיויר: לפני האקטיבציה האחרונה מוסיפים לפלט הבלוק את הקלט המקורי ללא עיבוד. פעולת החישוב המבוצעת בבלוק מאוירת להלן.



מכיוון שפעולת הסכום היא גזירה, ניתן לחשב את הגרדיאנט לאחור דרך הבלוק. לא רק שחישוב זה אפשרי, הוא אף יכול להועיל לתהליך הלמידה: חיבור הדילוג מאפשר לגרדיאנט להתפשט לאחור ללא הפרעה במקרים שבהם הגרדיאנט של שאר הבלוק מתאפס. נראה זאת דרך חישוב מפורש, ולצורך כך נסמן את טנזור הפלט על רכיביו ב- $Y = (y_0, \dots, y_n)$, את טנזור הקלט ב- $X = (x_0, \dots, x_n)$ ובדומה את חישובי הביניים בבלוק ב-

$$Y1 = (y_{1,0}, \dots, y_{1,n}),$$

$$Y2 = (y_{2,0}, \dots, y_{2,n})$$

כעת, נניח שגרדיאנט פונקציית המחיר לפי פלט הבלוק חושב זה מכבר ויש לחשב אותו לאחור. כלומר הערכים $\frac{\partial C}{\partial y_k}$ ידועים לכל $k = 0, \dots, n$ ויש לחשב את $\frac{\partial C}{\partial x_m}$ לכל $m = 0, \dots, n$. נתחיל בשימוש ראשון

בכלל השרשרת,

$$\frac{\partial C}{\partial x_m} = \sum_{k=0}^n \frac{\partial C}{\partial y_k} \frac{\partial y_k}{\partial x_m}$$

נחשב את $\frac{\partial y_k}{\partial x_m}$ בתורו כך:

$$\frac{\partial y_k}{\partial x_m} = \frac{\partial \text{ReLU}(y_{2,k})}{\partial x_m} = \frac{\partial \text{ReLU}(y_{2,k})}{\partial y_{2,k}} \frac{\partial y_{2,k}}{\partial x_m} = 1\{y_{2,k} \geq 0\} \frac{\partial y_{2,k}}{\partial x_m}$$

ונזכיר ש־

$$1\{x \geq 0\} = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$

היא הנגזרת של $\text{ReLU}(x)$ אשר חישובנו בעבר. בשלב זה יש לשים לב ש־

$$y_{2,k} = y_{1,k} + x_k$$

לפי הגדרת הבלוק, ולכן

$$\frac{\partial y_{2,k}}{\partial x_m} = \frac{\partial (y_{1,k} + x_k)}{\partial x_m} = \frac{\partial y_{1,k}}{\partial x_m} + \frac{\partial x_k}{\partial x_m} = \begin{cases} \frac{\partial y_{1,k}}{\partial x_m} & k \neq m \\ \frac{\partial y_{1,k}}{\partial x_m} + 1 & k = m \end{cases}$$

מכאן שגם אם הנגזרות דרך מסלול החישוב העליון בבלוק (ראו האיור לעיל) מתאפסות, קרי לכל $k \neq m$

$$\frac{\partial y_{1,k}}{\partial x_m} = 0$$

עדיין מתקיים

$$\frac{\partial y_{2,k}}{\partial x_m} = \begin{cases} 0 & k \neq m \\ 1 & k = m \end{cases}$$

ובהצבה בחזרה בנוסחאות הקודמות מתקבל

$$\frac{\partial C}{\partial x_m} = \sum_{k=0}^n \frac{\partial C}{\partial y_k} \frac{\partial y_k}{\partial x_m} = \sum_{k=0}^n \frac{\partial C}{\partial y_k} \cdot \frac{\partial y_{2,k}}{\partial x_m} \cdot 1\{y_{2,k} \geq 0\} = \frac{\partial C}{\partial y_m} \cdot 1\{y_{2,m} \geq 0\}$$

החשוב בתוצאה זו אינו הערך עצמו, אלא שהוא אינו מתאפס: ללא חיבור הדילוג, התאפסות הגרדיאנט דרך שאר הבלוק הייתה גוררת $\frac{\partial C}{\partial x_m} = 0$, ולפיכך לכל פרמטר השייך לשכבה **הקודמת** לבלוק זה ברשת

היינו מקבלים, שוב לפי כלל השרשרת,

$$\frac{\partial C}{\partial w} = \sum_{m=0}^n \frac{\partial C}{\partial x_m} \frac{\partial x_m}{\partial w} = 0$$

משמעות הדבר היא שהלמידה בעזרת אלגוריתם מבוסס גרדיאנט עבור כל הפרמטרים הקודמים לכל הבלוק הייתה עוצרת.

בלוקים שיוריים ברשתות קונבולוציה

בעוד הדיון שלעיל מצביע על הרלוונטיות הכללית של חיבורי דילוג, ResNet עצמה היא עודנה רשת קונבולוציה, ולפיכך בבלוק השיורי המתאים לה יש שימוש בפעולות הקונבולוציה במקום האגרגציה הלינארית. ראו זאת בקטע הקוד שלהלן.


```

class ResBlockConv(nn.Module):
    def __init__(self, in_channels):
        super().__init__()
        self.relu = nn.ReLU()
        self.conv1 = nn.Conv2d(in_channels, in_channels, 3,
                                padding="same", bias=False)
        self.bn1 = nn.BatchNorm2d(in_channels)
        self.conv2 = nn.Conv2d(in_channels, in_channels, 3,
                                padding="same", bias=False)
        self.bn2 = nn.BatchNorm2d(in_channels)
    def forward(self, X):
        Y1 = self.conv1(X)
        Y1 = self.bn1(Y1)
        Y1 = self.relu(Y1)
        Y1 = self.conv2(Y1)
        Y1 = self.bn2(Y1)
        Y2 = Y1+X
        Y = self.relu(Y2)
        return Y

```

מלבד החלפת השכבות הלינאריות בפעולות קונבולוציה עם גרעין 3×3 וריפוד השומר על ממד תמונת הפלט כממד תמונת הקלט (עמודה אחת ושורה אחת של אפסים בכל צד במקרה זה), נוספו שני שינויים קטנים הנוגעים לשכבות ה־batch normalization:

1. בבלוק זה יש שימוש בשכבות נורמליזציה מיוחדות עבור קונבולוציות, המנרמלות כל ערוץ בנפרד (במקום כל פיקסל בשכבת BN קלאסית).
2. מכיוון ששכבות ה־BatchNorm2d לומדות פרמטר bias משלהן עבור כל ערוץ ומוסיפות אותו לכל הפיקסלים בערוץ לאחר הנרמול, אין צורך בפרמטר המבצע אותו חישוב בשכבת הקונבולוציה. לכן מועבר הפרמטר `bias=False` לבנאי השכבה.

באמצעות שימוש בריפוד ושמירת מספר ערוצי הקלט כמספר ערוצי הפלט, גודל הטנזור $Y1$ במעבר קדימה בבלוק זה זהה לגודל הטנזור המקורי X , וכך ניתן לסכום אותם בחיבור הדילוג. עם זאת, יש לזכור שמטרתן של שכבות הקונבולוציה היא ללמוד מספר הולך וגדל של מחלצי מאפיינים (גרעיני הקונבולוציה), תוך כדי הקטנת רזולוציית תמונת הקלט. לפיכך, בתכנון הרשת המלאה יש צורך בבלוק שיורי נוסף המאפשר גמישות זו. ראו את מימושו בקטע הקוד שלהלן.

```
class ResBlockDownSamp(nn.Module):
    def __init__(self, in_channels):
        super().__init__()
        out_channels = in_channels*2 #

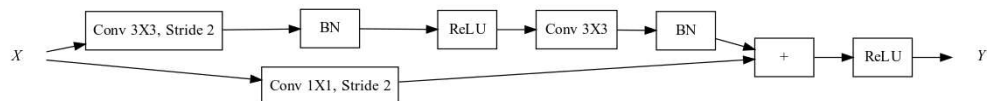
        self.relu = nn.ReLU()
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3,
                                padding=1, stride=2, bias=False) #

        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3,
                                padding="same", bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)

        self.downsampX=nn.Conv2d(in_channels, out_channels, 1 #
                                   , stride=2, bias=False)

    def forward(self, X):
        Y1 = self.conv1(X)
        Y1 = self.bn1(Y1)
        Y1 = self.relu(Y1)
        Y1 = self.conv2(Y1)
        Y1 = self.bn2(Y1)
        Y2 = Y1 + self.downsampX(X) #
        Y = self.relu(Y2)
        return Y
```

השינויים בין בלוק זה לבלוק הקודם מסומנים ב"#", והם נועדו להקטין את ממד האורך והרוחב פי חצי, בעוד מספר הערוצים גדל פי שניים, וזאת בעזרת שימוש בגודל פסיעה של 2. באיור,



נתבונן בממדי הטנזורים של חישובי הביניים בבלוק זה כדי להבין את השימוש בשכבת הקונבולוציה השלישית, `downsampX`: עבור טנזור קלט X בגודל $N \times C \times H \times W$, לאחר הקונבולוציה הראשונה גודל המשתנה $Y1$ יהיה $N \times 2C \times \frac{H}{2} \times \frac{W}{2}$ (בהנחה שממדי האורך והרוחב היו זוגיים), וכך גם לאחר הקונבולוציה השנייה, אשר משמרת את הממדים. כעת ברצוננו לסכום את תוצאת הביניים עם הקלט כדי להוסיף את חיבור הדילוג, אך ממדיהם שונים. לכן טנזור הקלט מוזן לתוך קונבולוציה עם גרעין 1×1 , לצורך הכפלת מספר הערוצים וחציית ממד האורך והרוחב. שאר החישוב דומה לזה המבוצע בבלוק הקודם.

רשתות שיריות (ResNets)

בעוד לעיל דנו ב-ResNet כאילו מדובר ברשת אחת, למעשה זהו מתכון לתכנון כמה רשתות בעלות עומק משתנה. לנוחותנו ניתן לטעון אותן ישירות דרך הספרייה PyTorch:

```
import torchvision.models as models
resnet18 = models.resnet18()
print(resnet18)
```

בהרצת קטע קוד זה יוגדר אובייקט רשת ResNet בעלת 18 שכבות ומבנה השכבות שלה יודפס. מהפלט ניתן לראות שהיא מורכבת מסדרה של בלוקים שיוריים רגילים המחוברים לבלוקים שיוריים המגדילים את מספר הערוצים. ההבדל בין רשת זו לבין רשתות ResNet אחרות, כגון ResNet50, הוא מספר הבלוקים בסדרה בלבד.

מעניין להתבונן בשכבות האחרונות ברשת, שאליהן מוזן פלט מחלץ המאפיינים המבוסס על שכבות הקונבולוציה:

```
last_layers = list(resnet18.children())[-2:]
print(*last_layers, sep="\n")

AdaptiveAvgPool2d(output_size=(1, 1))
Linear(in_features=512, out_features=1000, bias=True)
```

פלט:

בשכבה האחרונה 1,000 נוירונים, כצפוי מרשת אשר מיועדת לסווג את תמונות ImageNet ל-1,000 מחלקות שונות. פלט זה יוזן לפונקציית softmax ליצירת הסתברויות השייכות למחלקות. השכבה הקודמת לה מחשבת ממוצע עבור כל אחד מ-512 הערוצים בפלט סדרת הקונבולוציות על פני ממד האורך והגובה. בעזרת שכבה זו אפשר להזין לרשת קלט אשר אינו בדיוק הגודל הצפוי (תמונות 224×224): בהתאם לגודל התמונה המוזנת לרשת, אורך ורוחב הפלט של מחלץ המאפיינים ישתנה, אך מספר הערוצים יישאר 512 והשכבה האדפטיבית תתאים את הממדים לאלו הדרושים לשכבת הסיווג. כך יהיה ניתן להשתמש ברשת המאומנת גם עבור תמונות ברזולוציה שונה מהמקורית.

שאלות לתרגול

1. הסבירו מדוע ברירת המחדל של בלוק שיורי היא לחשב את פונקציית הזהות, כאשר הוא לא תורם להורדת פונקציית המחיר. התייחסו לתהליך אימון שבו משתמשים ברגולריזציה של הפרמטרים.
2. מנו את הפרמטרים ב"ראש הסיווג" המופיע בסוף ResNet18. השוו מספר זה למספר הפרמטרים במסווג של AlexNet.